

Wipro Capstone Project Documentation

1. Selenium Automation Testing Project

Project Title

Nykaa Health & Wellness Selenium Automation Testing Framework

Project Description

This project was developed using Selenium WebDriver with Python and the Pytest framework to automate the end-to-end testing process of the Nykaa Health & Wellness website. The main objective of the project was to reduce manual testing effort, improve testing accuracy, and ensure that important functionalities of the website work properly under different scenarios.

The automation framework follows the Page Object Model (POM) design pattern, which helps in organizing the code in a structured and reusable manner. This approach improves maintainability and makes the framework easier to scale for future enhancements.

The automation flow includes:

- Launching browser
- Searching products
- Adding products to cart
- Validating cart functionality
- Handling dynamic UI elements
- Capturing screenshots on failures
- Generating reports

The framework is designed to perform automated regression and smoke testing efficiently.

Test Strategy

The testing strategy used in this project includes:

Functional Testing

- Validation of search functionality
- Validation of add-to-cart functionality
- Validation of cart page behavior

End-to-End Testing

- Complete product purchase workflow testing

Regression Testing

- Ensuring previously working features remain unaffected after updates

Smoke Testing

- Basic validation of major functionalities before execution

Automation Framework Design

- Page Object Model (POM)
- Reusable utility functions
- Config-driven execution
- Screenshot capture on failures

Test Environment

Component	Details
Operating System	Windows 11
Programming Language	Python
Automation Tool	Selenium WebDriver
Testing Framework	Pytest
IDE	PyCharm
Browser	Google Chrome
Driver Manager	webdriver-manager
Reporting Tool	Allure Report
Version Control	Git & GitHub

Test Execution

Steps to Execute

1. Open project in PyCharm
2. Activate virtual environment
3. Install dependencies

4. Run pytest command

Command Used

```
pytest -v
```

For Allure Report

```
pytest --alluredir=allure-results  
allure serve allure-results
```

Test Output

The automation execution performs:

- Successful browser launch
- Product search execution
- Product addition to cart
- Cart validation
- Screenshot generation on failures
- Console execution logs

Sample Output:

```
PASSED tests/test_search.py  
PASSED tests/test_cart.py
```

Test Reports

The project generates:

Pytest HTML Reports

- Execution status
- Pass/Fail summary
- Execution logs

Allure Reports

- Detailed execution dashboard
- Step-by-step execution details

- Screenshots on failure
 - Execution timeline
 - Test statistics
-

2. BDD Automation Testing Project

Project Title

Nykaa Health & Wellness BDD Automation Framework using Behave

Project Description

This project was developed using Python, Selenium, and the Behave framework to implement Behavior Driven Development (BDD) methodology. The project focuses on writing test scenarios in a simple and readable format so that both technical and non-technical users can understand the testing workflow easily.

The framework automates user scenarios written in Gherkin language and validates the functionality of the Nykaa Health & Wellness website.

The project follows:

- Page Object Model (POM)
- BDD approach using feature files
- Reusable step definitions
- Automated report generation

The framework improves readability and collaboration between technical and non-technical stakeholders.

Test Strategy

The BDD testing strategy includes:

Scenario-Based Testing

- User scenarios written in Given-When-Then format

Functional Testing

- Product search validation
- Add to cart validation
- Cart page validation

End-to-End Testing

- Complete shopping flow validation

Regression Testing

- Repeated execution of scenarios after updates

BDD Framework Design

- Feature files
 - Step definitions
 - Hooks implementation
 - Reusable page classes
-

Test Environment

Component	Details
Operating System	Windows 11
Programming Language	Python
Automation Tool	Selenium WebDriver
BDD Framework	Behave
IDE	PyCharm
Browser	Google Chrome
Driver Manager	webdriver-manager
Reporting Tool	Allure Report
Version Control	Git & GitHub

Test Execution

Steps to Execute

1. Open project in PyCharm
2. Activate virtual environment
3. Install dependencies
4. Execute Behave command

Command Used

```
behave
```

For Allure Report

```
behave -f allure_behave.formatter:AllureFormatter -o allure-results  
allure serve allure-results
```

Test Output

The execution performs:

- Browser launch
- Execution of BDD scenarios
- Product search validation
- Add to cart validation
- Screenshot generation on failure
- Console execution logs

Sample Output:

```
1 feature passed  
3 scenarios passed  
15 steps passed
```

Test Reports

The project generates:

Behave Reports

- Scenario execution results
- Pass/Fail status
- Execution logs

Allure Reports

- Scenario-wise execution details
- Step execution visualization
- Failure screenshots

- Test execution analytics
 - Execution summary dashboard
-

Advantages of the Frameworks

Selenium Framework Advantages

- Reusable automation components
- Faster execution compared to manual testing
- Better regression coverage
- Easy maintenance using Page Object Model
- Detailed reporting support
- Scalable framework structure

BDD Framework Advantages

- Better readability using Gherkin syntax
 - Improved collaboration between teams
 - Easy understanding of business scenarios
 - Structured scenario-based testing
 - Reusable step definitions
 - Improved test traceability
-

Challenges Faced During Project Development

Selenium Project Challenges

- Dynamic element handling on Nykaa website
- Multiple tab handling
- Add-to-cart synchronization issues
- ChromeDriver compatibility issues
- XPath locator failures due to dynamic UI
- Cart popup handling

Solutions Implemented

- Explicit waits implementation
- Dynamic XPath creation
- Exception handling
- Screenshot capturing on failure
- WebDriver Manager integration
- Robust page object structure

BDD Project Challenges

- Behave step mapping issues
- Scenario parameterization
- Feature file organization
- Driver initialization through hooks
- Allure integration with Behave

Solutions Implemented

- Reusable step definitions
 - Hook implementation using environment.py
 - Proper feature organization
 - Shared driver management
 - Allure formatter integration
-

Framework Architecture

Selenium Framework Structure

```
Selenium_project/  
|  
├─ config/  
├─ pages/  
├─ tests/  
├─ utils/  
├─ reports/  
├─ logs/  
├─ data/  
└─ requirements.txt
```

Description of Components

config/

Contains framework configuration files such as browser settings and environment configuration.

pages/

Contains all page classes developed using Page Object Model.

tests/

Contains all test case files executed through Pytest.

utils/

Contains reusable utility functions such as screenshot utility, waits, and logging.

reports/

Stores generated reports and execution outputs.

logs/

Contains framework execution logs.

data/

Stores test data used during execution.

BDD Framework Structure

```
Nykaa_BDD/  
|  
├─ features/  
|   ├── steps/  
|   ├── environment.py  
|   └─ *.feature  
├─ pages/  
├─ utils/  
├─ reports/  
└─ requirements.txt
```

Description of Components

features/

Contains all feature files written in Gherkin syntax.

steps/

Contains step definition implementations for feature scenarios.

environment.py

Contains hooks implementation for setup and teardown activities.

pages/

Contains reusable page object classes.

utils/

Contains helper and utility functions.

reports/

Contains generated execution reports.

Tools and Technologies Used

Technology	Purpose
Python	Programming Language
Selenium WebDriver	Browser Automation
Pytest	Test Execution Framework
Behave	BDD Framework
Allure Report	Reporting
Git	Version Control
GitHub	Repository Hosting
PyCharm	IDE
webdriver-manager	Driver Management

Sample Test Scenario

Selenium Scenario

1. Launch browser
2. Open Nykaa website
3. Search for product
4. Select product

5. Add product to cart
 6. Open cart
 7. Validate cart item
 8. Close browser
-

BDD Scenario

Feature: Nykaa Product Search

Scenario: Add Product to Cart

Given User launches Nykaa website

When User searches for ON Whey Protein

And User adds product to cart

Then Product should be visible in cart

Test Reporting Details

Allure Reporting Features

- Dashboard view
- Test statistics
- Pass/fail trend
- Failure screenshots
- Execution timeline
- Step-by-step reporting
- Scenario execution details

Benefits of Reporting

- Easy debugging
 - Better test analysis
 - Quick failure identification
 - Improved reporting visibility
 - Professional report generation
-

Future Enhancements

Selenium Framework Enhancements

- Cross-browser execution
- Jenkins CI/CD integration
- Docker integration
- Data-driven framework enhancement
- Parallel execution support

BDD Framework Enhancements

- Scenario tagging optimization
 - Cloud execution support
 - API testing integration
 - Enhanced reporting dashboards
 - Continuous integration pipeline
-

Conclusion

Both Selenium and BDD automation frameworks were successfully developed for the Nykaa Health & Wellness application.

The successful implementation of both projects demonstrates:

- Automation testing implementation
- Page Object Model framework
- Selenium WebDriver usage
- Pytest and Behave integration
- Allure reporting
- GitHub version control integration
- End-to-End automation testing workflow

Overall, both automation frameworks helped improve software quality, reduce manual testing effort, and support efficient regression testing. The projects also provided practical experience in automation testing, framework development, reporting, and version control management using GitHub.